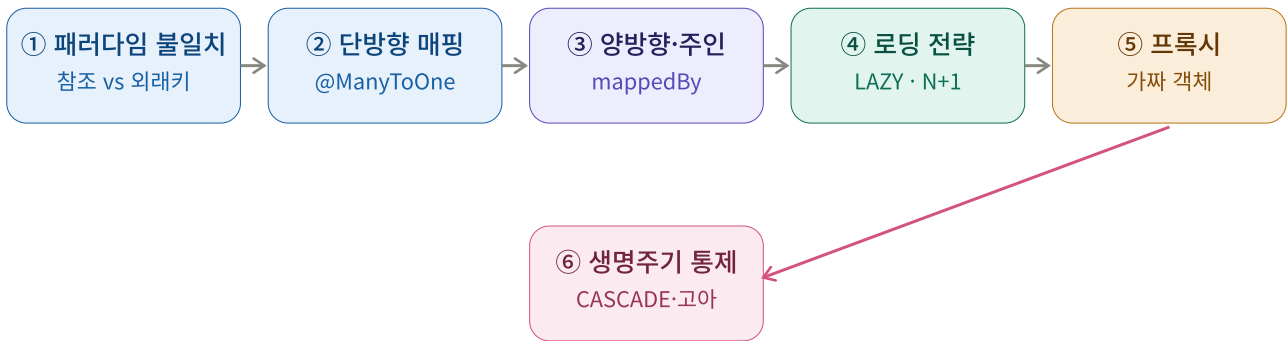


[Lv2] JPA 3주차 학습 노트

객체와 테이블의 조화: 연관관계 매핑과 생명주기 통제

JPA 학습의 가장 높은 산이자 실무 애러가 가장 많이 터지는 구간. "객체지향적으로 짰더니 왜 DB 저장이 안 되죠?"라는 질문에, 객체(참조)와 테이블(외래 키)의 패러다임 불일치를 **연관관계의 주인** 개념으로 풀고, 지연 로딩·프록시·N+1·CASCADE·고아 객체까지 정복한다.

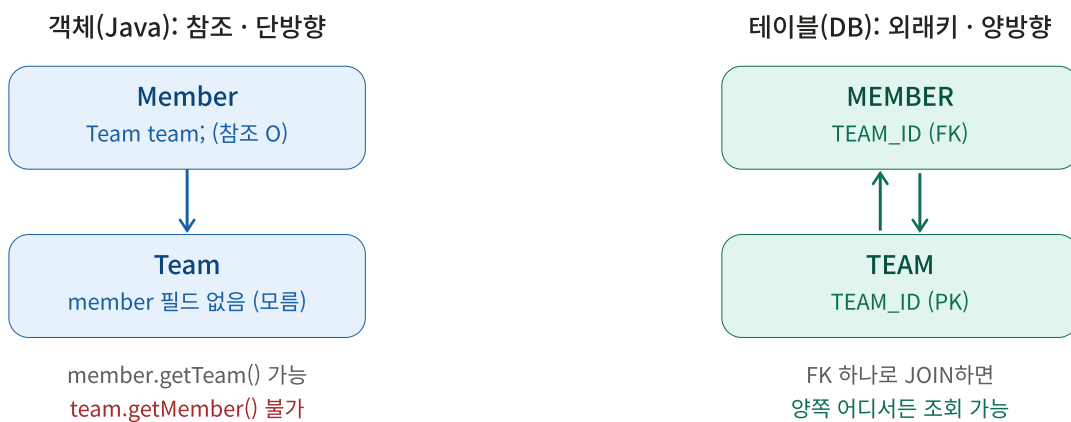


객체 세계와 테이블 세계를 JPA가 통역하고, 부모로 자식의 생명주기까지 지배한다

▲ 3주차 전체 흐름

① 패러다임 불일치 — "객체는 참조, 테이블은 외래키"

연관관계 매핑이 어려운 건 문법 때문이 아니라, **객체 지향 언어(Java)**와 관계형 DB(RDB)가 데이터를 연결하는 방식이 근본적으로 다르기 때문이다.



▲ 객체는 참조가 있는 쪽만 상대를 아는 단방향, 테이블은 FK 하나로 양방향

객체 참조가 좋은 이유 — 객체 그래프 탐색. 참조가 있으면 점(.)을 찍어 물 흐르듯 이동한다: `order.getMember().getName()`. 참조가 없으면 매번 ID를 꺼내 DB를 다시 뒤져야 한다(`em.find()` 반복). 하지만 **DB는 참조(메모리 주소)를 저장 못 하고 값(FK)만 저장한다.** 이 딜레마를 해결하는 통역사가 JPA다.

🔥 테이블 중심 설계의 비극

객체에 `Team team` 대신 `Long teamId` (FK 숫자)를 그대로 들고 있으면: ①객체 그래프 탐색 불가(`member.getTeam()` 끊김) ②DB 의존적 코드(`em.find()` 두 번) ③유지보수 최악. → 그냥 **데이터 덩어리**가 되어 객체지향의 장점을 다 잃는다.

JPA의 해결책: 매핑(Mapping)

"개발자는 객체(참조)를 써라. DB 저장(FK)은 내가 알아서 할게." — `teamId` 같은 식별자 대신 **객체 그 자체**를 품는다.

```
@Entity
public class JpaMember {
    @Id @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)    // 객체는 Team 자체를 참조
    @JoinColumn(name = "TEAM_ID")         // DB의 FK 컬럼(TEAM_ID)과 매핑
    private JpaTeam team;
}

// 저장: ID 안 꺼내고 객체 통째로 넘김 → JPA가 team의 PK를 TEAM_ID(FK)로 저장
member.setTeam(team);
// 조회: em.find() 없이 참조로 바로! (객체 그래프 탐색)
String teamName = findMember.getTeam().getName();
```

② 단방향 연관관계 — @ManyToOne으로 시작

단방향의 핵심은 "짜사랑": 회원은 팀을 알지만 팀은 회원을 모른다. 가장 중요한 건 "누가 외래 키(FK)를 가지고 있는가?" — DB에서 FK는 N(다) 쪽인 **MEMBER** 테이블에 있으므로, 매핑도 **Member** 에서 한다.

```
@Entity
public class UniMember {
    @Id @GeneratedValue @Column(name = "MEMBER_ID")
    private Long id;

    // @ManyToOne: "저는 N(다)이고 team은 1(일)입니다"
    // LAZY: 회원 조회 시 팀은 안 가져옴 (성능 최적화, 실무 필수)
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "TEAM_ID")    // DB의 UNI_MEMBER 테이블 FK 컬럼명!
    private UniTeam team;
}

@Entity
public class UniTeam {
    // 상대방: 회원을 모름 (필드 자체가 없음)
    @Id @GeneratedValue @Column(name = "TEAM_ID")
    private Long id;
    private String name;
}
```

[Deep Dive] `@JoinColumn(name="TEAM_ID")` 의 진짜 의미: UniTeam의 id 필드 이름이 아니라, 지금 작성 중인 UniMember 테이블의 FK 컬럼 이름이다. "INSERT 할 때 team 필드에 든 UniTeam의 PK를 꺼내 UNI_MEMBER 테이블의 TEAM_ID 컬럼에 저장해줘"라는 뜻.

왜 단방향만 먼저? 실무의 90%는 단방향으로 해결된다. "설계는 단방향으로 끝내라"는 격언이 있을 정도. 양방향은 반대쪽 조회가 꼭 필요할 때 추가해도 늦지 않다. 어노테이션 2개(`@ManyToOne` + `@JoinColumn`)만 기억하면 저장·조회·수정이 다 된다.

③ 양방향 연관관계와 "주인의 법칙"

"팀 페이지에서 소속 팀원 명단을 출력해줘" 같은 요구가 오면, 팀 입장에서 회원들을 볼 '창문'을 뚫는 게 양방향이다. **충격적 진실: 양방향을 해도 DB 테이블은 안 변한다.** DB는 원래 FK 하나로 양쪽을 다 조회할 수 있었다. 변하는 건 오직 객체(Java)뿐.

```
@Entity
public class BiTeam {           // 가짜/거울 (주인 아님)
    // mappedBy="team": "나는 주인이 아니다. BiMember의 team 필드가 진짜다"
    // 여기에 값 넣어도 DB의 FK는 안 변함. 오직 조회용!
    @OneToMany(mappedBy = "team")
    private List<BiMember> members = new ArrayList<>(); // NPE 방지 초기화
}

@Entity
public class BiMember {        // 👑 연관관계의 주인 (FK 관리)
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "TEAM_ID")
    private BiTeam team;
}
```

연관관계의 주인(Owner)이란?

한마디로 "DB 외래 키(FK)를 INSERT/UPDATE 할 수 있는 권한을 가진 엔티티". 양방향이면 참조가 2개(`Member.team` , `Team.members`) 인데 DB의 FK는 1개뿐. 둘 다 권한을 주면 충돌하므로, **진짜 리모컨(주인)** 하나를 정하고 나머지는 **읽기 전용(거울)**으로 만든다.

👑 **주인: Member.team**
@JoinColumn (FK 보유)
여기 값 넣어야 DB에 반영됨
`member.setTeam(team)`
= 진짜 리모컨

거울: Team.members
`mappedBy="team"` (읽기 전용)
여기 값 넣어도 DB는 무시
`team.getMembers().add()`
= 장난감 리모컨

▲ 두 개의 리모컨, 하나의 TV(DB의 FK 컬럼) — 주인만 채널을 바꿀 수 있다

🔑 **주인을 정하는 절대 규칙 (Golden Rule)** 외래 키(FK)를 가진 엔티티(보통 N쪽)가 무조건 주인. 이유: ①내 테이블의 내 컬럼을 내가 관리하는 게 자연스럽고 ②추가 UPDATE 쿼리가 안 나가 성능이 좋으며 ③1:N에서 N쪽이 무조건 FK를 가지기 때문. "N 쪽이 무조건 주인"이라고 외워도 무방.

🚨 **주인을 잘못 알면 벌어지는 참사** 주인이 아닌 `team.getMembers().add(member)` 에만 값을 넣으면 → DB에 MEMBER는 저장되지만 **TEAM_ID(FK)가 NULL**이 된다! 거울(mappedBy)은 JPA가 저장 시 철저히 무시하기 때문. 반드시 주인 `member.setTeam(team)` 을 호출해야 한다.

연관관계 편의 메서드 (양쪽 동기화)

DB 관점에선 주인에만 값을 넣으면 되지만, **객체(순수 메모리) 관점에선 틀렸다.** flush/clear 전 순수 객체 상태에서 한쪽에만 넣으면 반대쪽 컬렉션이 비어 로직 버그가 생긴다. 그래서 양쪽에 값을 넣는 로직을 하나로 묶는다.

```

public void changeTeam(BiTeam team) { // 단순 setter가 아니므로 이름 강조
    if (this.team != null) { // 1. 기존 팀에서 나를 제거 (유령 회원 방지)
        this.team.getMembers().remove(this);
    }
    this.team = team; // 2. 주인에 값 설정 (DB FK 저장용)
    if (team != null) { // 3. 반대쪽 리스트에도 추가 (객체 동기화)
        team.getMembers().add(this);
    }
}
}

```

⚠ 양방향의 무한 루프 함정 Lombok `@ToString` / `@Data` 를 그냥 쓰면 Member↔Team이 서로 `toString()` 을 무한 호출해 `StackOverflowError` . → 연관관계 필드 제외(`@ToString(exclude="team")`). API 응답으로 엔티티를 직접 반환하지 말고 반드시 DTO로 변환(Jackson JSON 무한 루프 방지).

④ 다양한 연관관계 & 로딩 전략

실무 매핑 전략 (Golden Rule)

관계	전략	이유
기본	무조건 <code>@ManyToOne</code> 단방향으로 시작	테이블 구조와 일치, INSERT 한 방, 복잡도 최저
1:N 단방향	<code>@OneToOne</code> 만 → 쓰지 마라	남의 테이블(MEMBER)에 불필요한 UPDATE 쿼리 발생, 헛갈림
양방향 필요 시	<code>@ManyToOne</code> + 반대쪽 <code>mappedBy</code> 추가	테이블 변경 없이 코드 몇 줄로 역방향 조회 확보
1:1	주 테이블(자주 조회하는 쪽)에 FK	프록시·지연로딩이 안먹 동작 (대상 테이블에 FK 두면 LAZY가 강제 EAGER화)
N:M	<code>@ManyToMany</code> 절대 금지 → 연결 엔티티로	중간 테이블이 숨겨져 수량·날짜 등 추가 컬럼을 못 넣음

☞ `@ManyToMany`는 "달콤한 독사과" 처음엔 중간 엔티티 없이 리스트로 바로 연결돼 편하지만, 기획자가 "주문 수량·날짜도 보여주세 요" 하는 순간 끝. 숨겨진 중간 테이블에 컬럼을 추가할 방법이 없다. → 중간 테이블을 **당당한 연결 엔티티(OrderItem)**로 승격시켜 1:N + N:1로 풀어라.

```

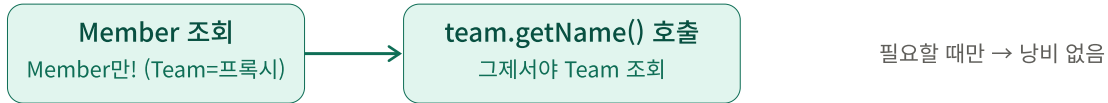
@Entity
public class MemberProduct { // 연결 엔티티 (실무선 OrderItem 등)
    @Id @GeneratedValue @Column(name = "ORDER_ITEM_ID")
    private Long id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "MEMBER_ID")
    private ManyMember member;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "PRODUCT_ID")
    private ManyProduct product;
    private int orderCount; // 중간 테이블이 노출되어 추가 정보 자유롭게!
    private LocalDateTime orderDate;
}

```

즉시 로딩(EAGER) vs 지연 로딩(LAZY)

"회원을 조회하면 팀도 가져와야 할까?" — **EAGER**(즉시): Member 조회 시 Team도 JOIN해서 즉시. **LAZY**(지연): Member만 가져오고 Team은 실제 사용(`team.getName()`) 시점에 조회. 카페 비유로, EAGER는 안 먹을 머핀까지 세트로 주는 것, LAZY는 필요할 때 주문하는 것.

LAZY (지연) — 추천



EAGER (즉시) — 실무 금지



▲ LAZY는 필요할 때만, EAGER는 무조건 다 — 실무는 무조건 LAZY

⚠ 기본값의 함정 (암기법: 뒤에 뭐가 오나)

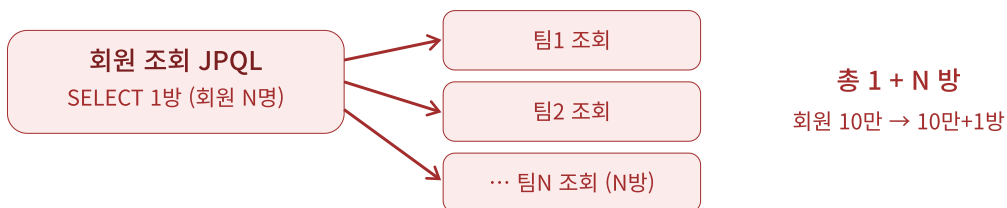
어노테이션 뒤쪽(After)이 **One**이면 **EAGER**, **Many**면 **LAZY**가 기본값.

어노테이션	뒤쪽	기본값
@ManyToOne	One	EAGER (위험!)
@OneToOne	One	EAGER (위험!)
@OneToMany	Many	LAZY
@ManyToMany	Many	LAZY

실무에서 가장 많이 쓰는 @ManyToOne 이 하필 EAGER가 기본이므로, 기계적으로 **fetch = FetchType.LAZY** 를 붙이는 습관이 필수다.

💣 N+1 문제 (가장 중요)

`em.find()` 는 JPA가 JOIN으로 최적화하지만, JPQL(`select m from Member m`)은 작성된 SQL 그대로 번역된다. EAGER면 회원 100명 조회 쿼리 1방 후, "즉시 로딩이니 팀도 다 채워야지" 하며 팀 조회 쿼리 100방이 추가로 나간다(총 N+1방). 회원이 10만이면 쿼리 10만+1방 → 서버 다운.



💡 해결: LAZY 기본 + 필요할 때만 Fetch Join

`select m from Member m join fetch m.team` → INNER JOIN 쿼리 딱 1방으로 해결

▲ N+1 문제와 Fetch Join 해결

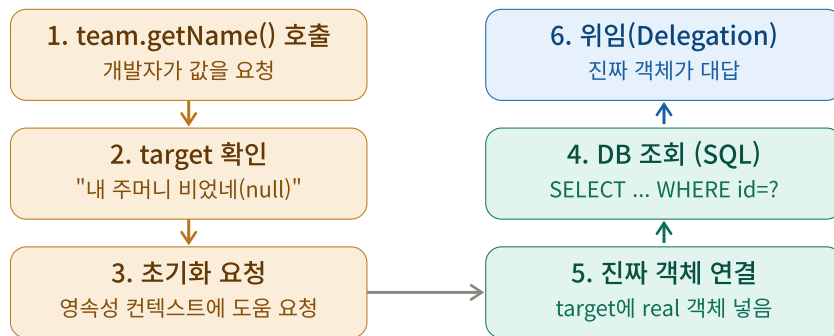
✅ **실무 정석** 기본은 무조건 LAZY로 안전하게 두고, 여러 건을 함께 가져와야 하는 특정 기능에서만 **Fetch Join**(`join fetch m.team`)으로 쿼리 1방에 묶어 가져온다. 성능과 안전을 모두 잡는 방법.

⑤ 프록시(Proxy) — 지연 로딩을 가능하게 하는 "가짜 객체"

프록시는 "진짜 객체인 척하면서 흐름을 제어하는 가짜 객체". 비유는 **연예인 매니저** — 주최자(Client)는 매니저(Proxy)에게 연락하고, 매니저는 사전 처리 후 진짜 가수(Target)에게 노래를 위임한다. 매니저는 노래를 못 하고, 진짜를 데려오는 역할만 한다.

비유(엔터)	개발 용어	설명
행사 주최자	Client (Service)	기능을 사용하려는 쪽
매니저	Proxy (\$Proxy...)	고객을 직접 상대하는 대리인
아이유(본체)	Target (진짜 Entity)	실제 로직/데이터를 가진 객체

프록시의 특징: ①실제 엔티티를 상속받아 만들어짐(겉모습 동일) ②내부에 진짜 객체 참조(`target`) 보관 ③처음엔 target이 비어있고, 실제 값을 요청(`getName()`)하면 그때 DB 조회로 채움. 이 과정이 **프록시 초기화**.



3번에서 영속성 컨텍스트가 "퇴근"했으면 → **LazyInitializationException!**

지연 로딩은 영속성 컨텍스트(트랜잭션)가 살아있을 때만 동작

▲ 프록시 초기화 6단계

⚠ 오해 금지 & 타입 비교 함정

초기화해도 **프록시는 끝까지 프록시**(진짜 객체로 교체 X, 내부 target 연결만). 그래서 프록시는 원본을 **상속한 자식**이라 `ref.getClass() == Team.class` 는 **false**. 반드시 `instanceof` 를 써야 한다. `equals()` 도 필드 직접 접근(`member.name`) 대신 **Getter**(`getId()`)를 써야 초기화가 유도됨.

LazyInitializationException 해결책 비교

방법	추천 상황	장점	단점
Fetch Join	기본 추천 ★	쿼리 1방, 가장 깔끔	쿼리 복잡해질 수 있음
DTO 조회	API·성능 중요	필요한 데이터만, 예외 원천 차단	DTO 클래스 작성 필요
OSIV (true)	어드민·소규모	개발 편함, 에러 안 남	대용량 트래픽서 커넥션 고갈 → 서버 장애

강제 초기화	급한 땀땀	빠른 수정	쿼리 2번 (N+1 위험)
--------	-------	-------	----------------

OSIV(Open Session In View): 영속성 컨텍스트의 생명주기를 트랜잭션이 아닌 **HTTP 요청 전체**로 확장. 스프링 부트 기본값이 **true**라 컨트롤러에서도 지연 로딩이 되지만, DB 커넥션을 오래 붙잡아 트래픽 많으면 위험. **실무 정석: OSIV를 끄고(false) Service 안에서 Fetch Join이나 DTO로 다 가져와 나온다.**

⑥ 생명주기 통제 — CASCADE & 고아 객체

영속성 전이 CASCADE — "엄마가 가면 나도 간다"

부모를 저장/삭제할 때 연관된 자식도 함께 처리하는 옵션. CASCADE가 없으면 자식마다 `em.persist()` 를 일일이 호출해야 하지만, `cascade = CascadeType.ALL` 을 걸면 부모만 `persist`해도 자식이 자동 저장된다.

```
@Entity
public class CascadeParent {
    @OneToMany(mappedBy = "parent", cascade = CascadeType.ALL)
    private List<CascadeChild> children = new ArrayList<>();

    public void addChild(CascadeChild child) {    // 편의 메서드
        children.add(child);
        child.setParent(this);
    }
}
// em.persist(parent) 한 줄 → INSERT 부모 1 + 자식 2 = 총 3방 자동 실행
```

옵션	설명	추천 상황
ALL	모든 상태 변화(저장·삭제·병합 등) 전파	가장 무난. 부모-자식 생명주기 일치 (게시글-첨부파일)
PERSIST	저장만 전파 (부모 지워도 자식 남음)	로그성 데이터 (탈퇴해도 가입로그 보존)
REMOVE	삭제만 전파	부모 사라지면 무의미한 데이터 (댓글, 좋아요)

CASCADE 대형 사고 시나리오 (해시태그 삭제 사건)

Post의 tags에 `CascadeType.ALL` 을 걸고 스팸 게시글 하나를 삭제 → JPA가 "태그도 지워달라네?" 하며 **#Java 태그 자체를 DB에서 삭제** → 그 태그를 달던 수천 개의 정상 게시글이 태그를 잃는 장애. 복구 불가.

✅ 자가 진단 체크리스트

① 자식이 부모와 생명주기(생성/삭제)를 같이 하는가? (아니오 → 금지) ② 자식을 참조하는 곳이 **오직 이 부모 하나뿐**인가? (예 → ALL OK / 아니오 → 금지). "애매하면 쓰지 마라. 안 써서 불편한 건 코드 몇 줄이면 되지만, 써서 데이터가 날아가면 복구 못 한다."

고아 객체(Orphan Removal) — "부모 잃은 자식은 삭제된다"

`orphanRemoval = true` 를 켜면, `delete()` 호출 없이 **자바 리스트에서 자식을 빼기만 해도** DB에서 DELETE된다. 폴더에서 파일을 빼면 갈 곳이 없어 삭제되는 것과 같다.

```
@OneToMany(mappedBy = "parent", orphanRemoval = true)
private List<Child> children = new ArrayList<>();
```

```
parent.getChildren().remove(0); // em.remove() 안 했는데
tx.commit();                  // → delete from Child where id=? 실행!
```

[Deep Dive] CascadeType.REMOVE vs orphanRemoval=true (면접 단골)

둘 다 "삭제"지만 삭제를 유발하는 조건이 다르다.

상황	CascadeType.REMOVE	orphanRemoval = true
부모 삭제 <code>em.remove(parent)</code>	자식도 삭제 (O)	자식도 삭제 (O)
리스트에서 뺄 <code>list.remove(child)</code>	삭제 안 됨 (X) — 관계만 끊김	자식 삭제 (O) — 고아 취급

비유: 자식을 집에서 쫓아냈을 때 — REMOVE는 자식이 집만 잃고 밖에서 살아있고, orphanRemoval은 갈 곳이 없어 사망한다.

🏆 **Aggregate Root 패턴** 실무선 보통 `cascade = ALL + orphanRemoval = true` 를 함께 켜다. 그러면 부모를 통해서만 자식의 생명주기를 완벽히 관리(add로 생성, remove로 삭제, 부모 삭제 시 전부 삭제) — DDD의 Aggregate Root 개념. 단, 자식이 "나만 소유하는 비밀스러운 존재"일 때만! (게시글-작성자 관계에 쓰면 해고각 🤖)

3주차 한 문장 요약

객체(참조)와 테이블(외래키)의 **패러다임 불일치**를 JPA가 매핑으로 통역한다. 저장은 무조건 **연관관계의 주인(FK 가진 N쪽)**에게, 설계는 **@ManyToOne 단방향**으로 끝내고 필요할 때만 양방향/Fetch Join을 더한다. 모든 연관관계는 **LAZY**로 깔아 N+1을 막고, 프록시는 끝까지 프록시임을 기억한다. 부모로 자식의 생명주기를 지배하되(**CASCADE·고아 객체**), 자식이 공유되는 존재면 절대 쓰지 않는다.

셀프 체크 (복습용)

Q1. 양방향 매핑에서 저장할 때 어느 쪽에 값을 넣어야 DB에 반영되나?

A. **연관관계의 주인(FK 가진 쪽, mappedBy 가 없는 쪽)**. 거울(mappedBy)에만 넣으면 FK가 NULL.

Q2. @ManyToOne 만 적으면 왜 위험한가?

A. 뒤가 One이라 기본값이 **EAGER**. JPQL에서 N+1 문제를 유발. 무조건 `fetch = LAZY` 명시.

Q3. N+1 문제의 근본 원인과 해결책은?

A. JPQL이 EAGER 설정을 보기 전에 SQL을 먼저 날려 발생. 해결은 LAZY 기본 + **Fetch Join**.

Q4. 초기화하면 프록시가 진짜 객체로 바뀌나?

A. 아니오. 끝까지 프록시(내부 target 연결만). 그래서 `==` 대신 `instanceof`.

Q5. `list.remove(child)` 했을 때 REMOVE와 orphanRemoval의 차이는?

A. REMOVE는 관계만 끊고 자식 생존, orphanRemoval은 고아로 보고 삭제.